

NBA Win Classification: Sports Capstone

Grant Robinson

Abstract: This project examines whether a team's recent performance can be used to predict NBA game outcomes more accurately than relying on season-long results or team reputation. Using 2025 NBA regular season box score data, I engineered leakage-safe 20-game rolling averages for eight key performance statistics, along with home court and rest day features. The 20-game window was selected by testing multiple rolling window sizes. Logistic Regression, Random Forest, and XGBoost were tuned using GridSearchCV and compared against a persistence baseline using accuracy, ROC-AUC, and statistical tests, including Bootstrap confidence intervals, Wilcoxon signed-rank, and McNemar's test. All three models outperformed the persistence baseline, showing that recent team performance contains meaningful predictive information. However, the original hypothesis was refuted. Unexpectedly, Logistic Regression significantly outperformed both Random Forest and XGBoost based on the bootstrap confidence intervals. These results suggest that, for this feature set, adding model complexity did not improve predictive performance. Instead, future improvements will likely come from incorporating better contextual features

I. Introduction

Problem statement : Predicting NBA game outcomes is challenging because team performance fluctuates throughout the season. Teams are often evaluated based on their season record or how good they look on paper instead of their recent performance trends.

RQ1: Can recent team performance trends accurately predict the outcome of an NBA game?

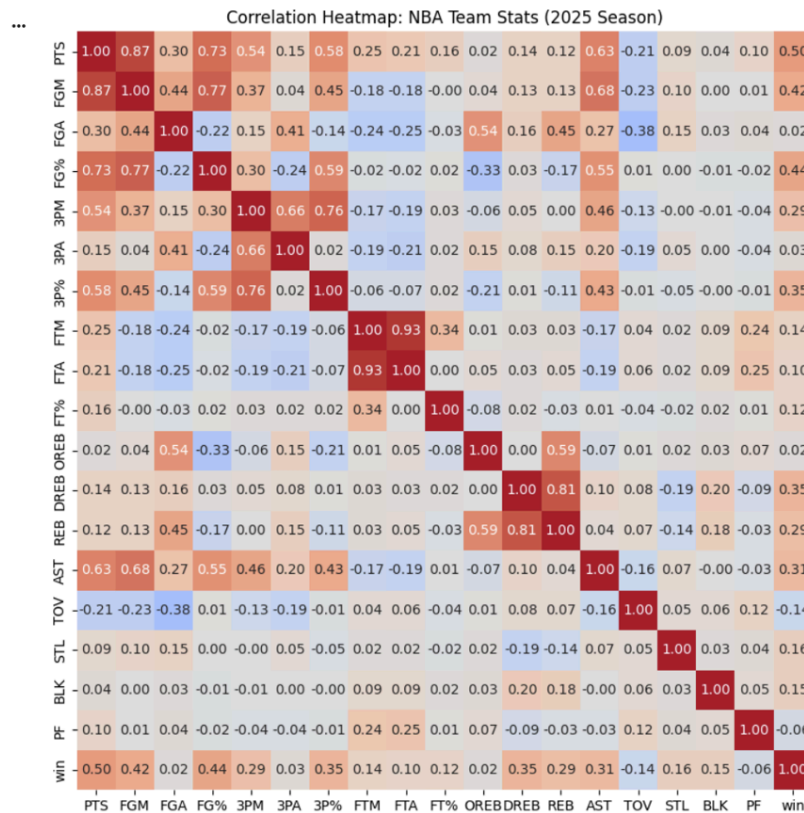
RQ2: Which Performance Metrics are most associated with winning NBA Games?

Hypothesis: More complex models, specifically Random Forest or XGBoost trained on the same leakage-safe rolling team statistics, will predict game outcomes more accurately than Logistic Regression

II. Data, Data Analysis, and Feature Engineering

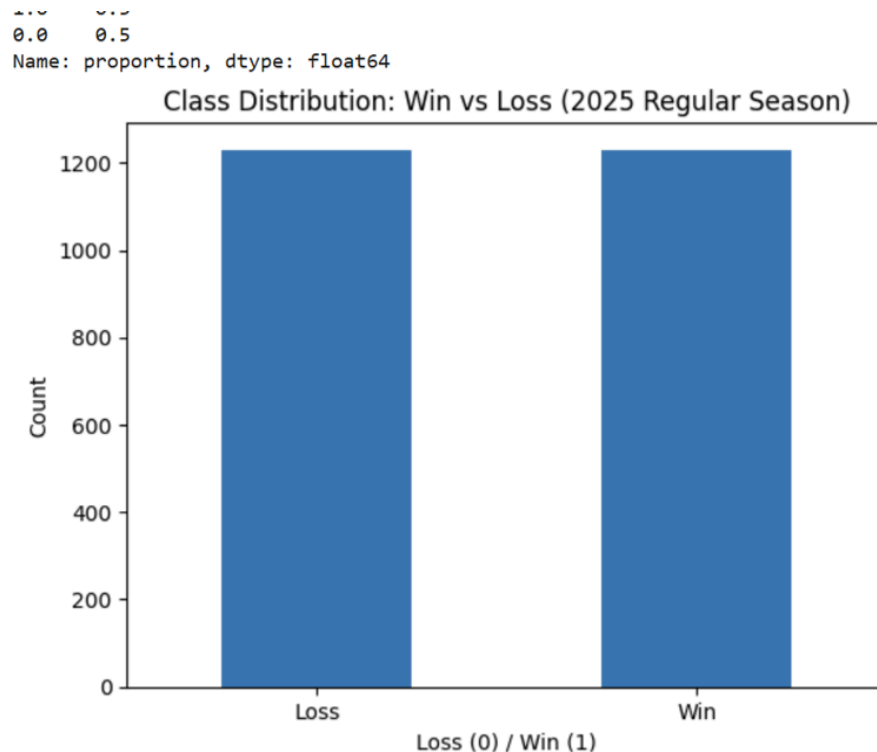
Dataset: For this project, I used a publicly available Kaggle dataset of NBA traditional box scores compiled from official NBA game logs. Although the dataset spans the 1997–2025 seasons, I filtered it to include only the 2025 regular season to provide a consistent snapshot without mixing different eras, rules, or team rosters. The final dataset contains 1,230 games (2,460 team-game observations, with one row per team per game) and 29 raw features, including points, shooting percentages, rebounds, assists, turnovers, steals, and blocks.

EDA: By exploring the data, I wanted to understand which raw performance statistics actually relate to winning before deciding what to build my features from



This heatmap provides an overview of the relationships among the raw box score statistics and their correlations with the target variable, win, shown in the bottom row. Several relationships are worth highlighting. Points exhibited the strongest positive correlation with winning (0.50), followed by field goal percentage (0.44) and defensive rebounds (approximately 0.35). In

contrast, turnovers showed the strongest negative correlation (-0.14), indicating that teams committing more turnovers were less likely to win. The heatmap also revealed strong correlation between field goals made and field goal percentage (0.77), reflecting their mathematical relationship. This redundancy informed me which raw statistics were selected as the basis for later feature engineering, rather than including every available raw variable.



This bar chart shows the distribution of the target variable, win versus loss, across the 2025 season. The classes are evenly balanced at 50% each because every game produces one winning team and one losing team. Therefore, this resulted in no resampling or class weighting because it wasn't necessary. The balanced classes also made accuracy an appropriate evaluation metric, since the model could not achieve high accuracy simply by favoring a majority class.

Data Preprocessing:

I found no missing values in the raw data. I also confirmed that the target variable, win, was perfectly balanced overall (50%/50%). After the chronological split, the training set remained nearly balanced at 50.1%/49.9%, while the test set was 50.4%/49.6%. Since a chronological split does not automatically preserve class balance, I verified the distribution separately in each set.

Because there was no meaningful class imbalance, I did not use resampling (oversampling or undersampling) or class weighting.

After creating the rolling average features, I removed each team's earliest games that did not have enough prior history to calculate a valid 20-game rolling average. This reduced the dataset from 2,460 to 2,400 usable observations. This was done to avoid invalid feature values and was not a response to missing data in the original dataset.

I standardized the numerical features using StandardScaler, fitting the scaler only on the training data and then applying it to the test data. This prevented information about the test set's distribution from influencing preprocessing and helped avoid data leakage. No categorical encoding was needed because home/away status was represented directly as a binary `is_home` feature during feature engineering.

I used a chronological 80/20 train-test split rather than a random split, with the earliest 80% of games used for training and the most recent 20% used for testing. This preserved the time-series structure and prevented future games from influencing predictions of earlier games. Within the training set, I used 5-fold TimeSeriesSplit cross-validation rather than standard k-fold cross-validation. This maintained the chronological order within each fold and prevented future games from being used to predict earlier games.

Feature Engineering:

I engineered eight leakage-safe rolling average features based on the core box score statistics that showed the strongest relationships with winning during EDA: points, field goal percentage, three-point percentage, rebounds, assists, turnovers, steals, and blocks. Each rolling average was calculated using a one-game shift before averaging, ensuring that a game's features were based only on information from previous games and never on the game itself or future games.

I tested rolling window lengths of 5, 10, 15, 20, and 25 games rather than assuming a single window was most appropriate. Each window was evaluated using an untuned Logistic Regression model, and the 20-game window was selected because it produced the highest accuracy and ROC-AUC.

I also created two contextual features. The `is_home` variable indicates whether a team was playing at home and captures home-court advantage, while `rest_days` measures the number of days since the team's previous game and captures potential scheduling and fatigue effects.

I did not apply formal dimensionality reduction, such as PCA, because the final feature set contained only 10 features. Instead, feature selection was based on what I noticed in the EDA correlation analysis

III. Methodology

Experiment Setup: I first created leakage-safe rolling features using five candidate window sizes: 5, 10, 15, 20, and 25 games. I then compared the windows and selected the 20-game window based on its performance. After selecting the window, I created the final 10-feature dataset and split the data chronologically into training and test sets. I then verified the class distribution, standardized the features using only the training data, and trained and tuned each model using 5-fold TimeSeriesSplit cross-validation. Finally, I evaluated the tuned models on the held-out test set and statistically compared their performance.

I used a chronological 80/20 train-test split, with 1,920 training observations and 480 test observations, rather than a random split. This preserved the time-series structure and prevented future games from being used to predict earlier games. Within the training data, I used 5-fold TimeSeriesSplit cross-validation for hyperparameter tuning and model stability. This was preferred over standard k-fold cross-validation because TimeSeriesSplit maintains the chronological order of the data. I used random_state=42 consistently for reproducibility. The project was completed using Python 3.12 with scikit-learn 1.8.0, pandas 3.0.2, NumPy 2.4.4, and XGBoost.

I compared four models, including a naïve baseline:

- Persistence baseline: Predicts a team's next outcome based on its previous game. If the team won its previous game, the model predicts a win. If they lose, it predicts a loss. This provides a simple benchmark for determining whether the machine learning models are learning useful patterns beyond recent results.
- Logistic Regression: Uses a weighted combination of the input features and a sigmoid function to produce a probability of winning. The model learns the feature weights during training to minimize prediction error.
- Random Forest: Builds multiple decision trees using different subsets of the data and features, then combines their predictions. This allows the model to capture non-linear relationships while reducing the risk of relying too heavily on any single tree.
- XGBoost: Builds decision trees sequentially, with each new tree focusing on correcting errors made by previous trees. This allows the model to progressively improve its predictions and capture more complex relationships.

Logistic Regression Formulas:

(Decision function)

$$z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_{10} X_{10}$$

(sigmoid Function)

$$P(Y = 1|X) = \frac{1}{1 + e^{-z}}$$

For the decision function, X_1 through X_{10} are the input features (the eight rolling statistics, `is_home`, and `rest_days`), β_1 through β_{10} are the learned coefficients determining how strongly and in which direction each feature influences the prediction, and β_0 is the intercept. This value z is passed through the sigmoid function. The sigmoid function is what converts z into a probability between 0 and 1. A prediction above 0.5 is classified as a win and below is a loss in my case.

Hyperparameter tuning: I used GridSearchCV to tune the hyperparameters for each model, using F1-score as the scoring metric.

For Logistic Regression, I tuned (`C`), `penalty`, and `class_weight`. The search included 11 values of (`C`) ranging from 0.001 to 100, four `penalty` options (L1, L2, `elasticnet`, and `none`), and two `class_weight` options (`None` and `balanced`).

- **C** controls the strength of regularization. Smaller values apply stronger regularization and can lead to underfitting, while larger values allow the model more flexibility and can increase the risk of overfitting.
- **Penalty** determines how regularization is applied. L1 can reduce some coefficients to zero, while L2 shrinks coefficients without removing them.
- **class_weight** changes how much the model penalizes errors for each class. Since the classes were already balanced, this parameter was not expected to have much effect.

For Random Forest, I tuned (`n_estimators`) (100, 200, 300), (`max_depth`) (3, 5, 7, 10, `None`), and (`min_samples_leaf`) (1, 3, 5, 10).

- **n_estimators** controls the number of trees in the forest. Adding more trees can make predictions more stable, although the benefit eventually levels off.
- **max_depth** controls how deep each tree can grow. Shallower trees reduce the risk of overfitting, while deeper trees can capture more complex patterns but may overfit.
- **min_samples_leaf** sets the minimum number of observations required in a leaf. Higher values make the trees less complex and can help prevent overfitting.

For XGBoost, I tuned (`n_estimators`) (100, 200, 300), (`max_depth`) (3, 5, 7), and (`learning_rate`) (0.01, 0.05, 0.1, 0.3).

- **learning_rate** controls how much each new tree contributes to the model. Lower values make smaller updates and generally require more trees, while higher values make larger updates but can increase the risk of overfitting.

For Logistic Regression, grid search selected $C=0.5$, $\text{penalty}='l2'$, and $\text{class_weight}='balanced'$ as the best-performing combination. For Random Forest, it selected $n_estimators=100$, $\text{max_depth}=5$, and $\text{min_samples_leaf}=5$. For XGBoost, it selected $n_estimators=100$, $\text{max_depth}=3$, and $\text{learning_rate}=0.01$.

..

	Model	Accuracy (Untuned)	Accuracy (Tuned)	ROC-AUC (Untuned)	ROC-AUC (Tuned)
0	Logistic Regression	0.621	0.621	0.664	0.664
1	Random Forest	0.565	0.610	0.601	0.646
2	XGBoost	0.542	0.615	0.545	0.651

When looking at the results above, tuning had no effect on Logistic Regression, with the tuned model producing the same accuracy and ROC-AUC as the untuned model. Random Forest showed the largest improvement, increasing from 0.565 to 0.610 in accuracy and from 0.601 to 0.646 in ROC-AUC. XGBoost also improved with tuning, but to a lesser extent, with accuracy increasing from 0.598 to 0.615 and ROC-AUC from 0.628 to 0.651.

Evaluation Methodology: My primary evaluation metric was ROC-AUC. It measures how well the model distinguishes wins from losses across different probability thresholds rather than relying on a single cutoff such as 0.5. I chose ROC-AUC because it provides a consistent way to compare the three different model types without prioritizing one type of error over another. Unlike a medical diagnosis task, where false negatives may be more costly than false positives, neither type of error has a clearly greater consequence in this project.

I also reported four supporting metrics to provide a more complete view of model performance:

- Accuracy: $((TP + TN)/(TP + TN + FP + FN))$, the proportion of predictions that were correct. Because the classes were balanced, accuracy was an appropriate metric for this dataset.
- Precision: $(TP/(TP + FP))$, the proportion of predicted wins that were actually wins.
- Recall: $(TP/(TP + FN))$, the proportion of actual wins that the model correctly identified.
- F1-score: $(2(\text{Precision} \times \text{Recall})/(\text{Precision} + \text{Recall}))$, the harmonic mean of precision and recall.

These additional metrics helped identify differences in model behavior that ROC-AUC alone would not show.

I used three statistical tests: Bootstrap confidence intervals, Wilcoxon signed-rank, and McNemar's test. The Bootstrap CI and Wilcoxon tests compared 5-fold cross-validated ROC-AUC scores, while McNemar's test compared predictions on the held-out test set.

Bootstrap confidence intervals were my primary test because they do not require a specific distributional assumption. I resampled the paired ROC-AUC differences 10,000 times to calculate a 95% confidence interval. I used the Wilcoxon signed-rank test instead of a paired t-test because only five cross-validation folds were available, making the t-test less appropriate with such a small sample. McNemar's test was used to compare disagreements between model predictions on the same test set, making it appropriate for evaluating paired classification results.

IV. Results and discussion

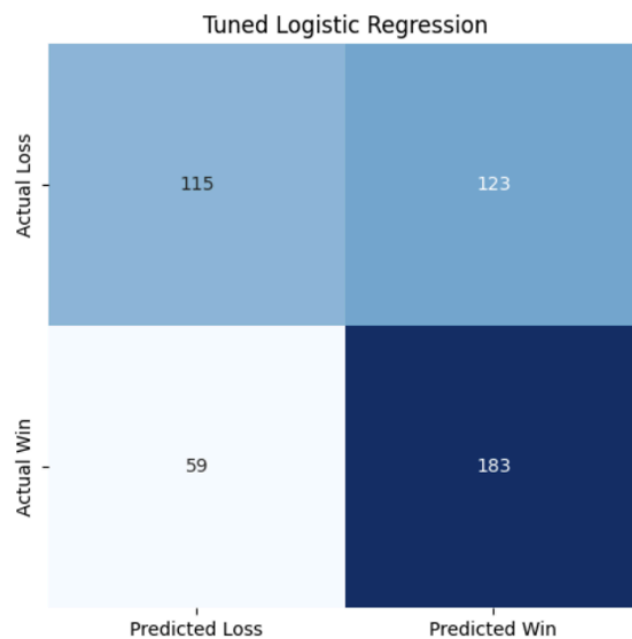
Model Comparison: I compared the tuned Logistic Regression model against others. The persistence baseline, untuned Logistic Regression, and two alternative models, Random Forest and XGBoost.

I used a persistence baseline, which predicts that a team will repeat its previous game's outcome, rather than a majority-class or random-guess baseline. Because the classes were balanced, those alternatives would provide little useful information, while persistence is more appropriate for a time-series problem. ROC-AUC was used as the primary metric, with accuracy, precision, recall, and F1-score reported as supporting measures.

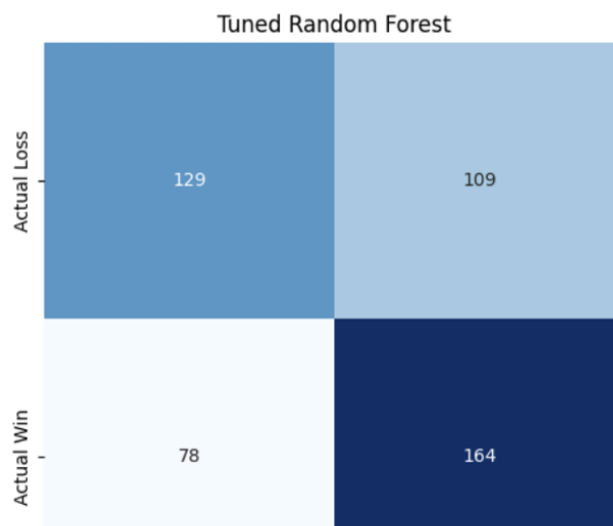
..

	Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
0	Persistence Baseline	0.586	0.586	0.588	0.587	NaN
1	Logistic Regression (Untuned)	0.621	0.598	0.756	0.668	0.664
2	Logistic Regression (Tuned)	0.621	0.598	0.756	0.668	0.664
3	Random Forest (Tuned)	0.610	0.601	0.678	0.637	0.646
4	XGBoost (Tuned)	0.615	0.600	0.707	0.649	0.651

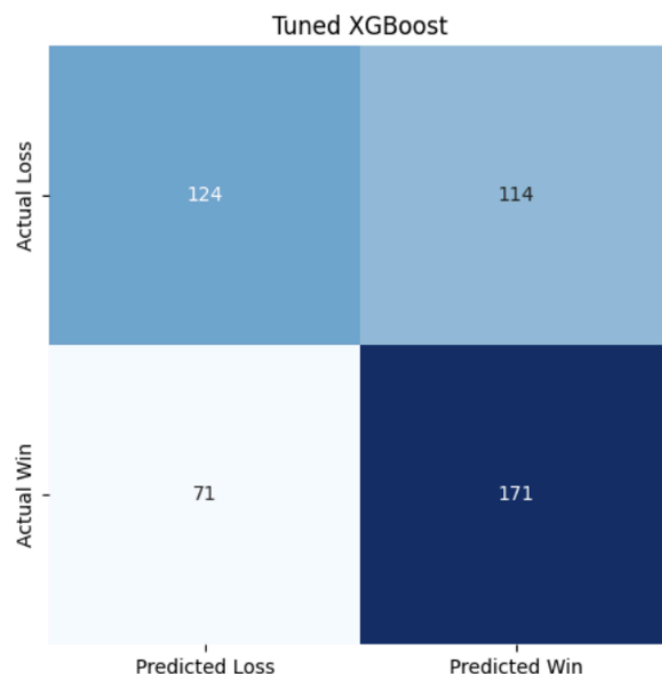
It is important to note that ROC-AUC could not be computed for the persistence baseline, since it produces only hard predictions, not probability scores. However, looking at the results All three tuned models outperformed the persistence baseline, showing that they captured useful predictive signals beyond simply repeating the previous outcome. Logistic Regression performed best, outperforming Random Forest and XGBoost across accuracy, recall, F1-score, and ROC-AUC. Tuning did not change Logistic Regression's results though, with the tuned and untuned models producing identical predictions.



Logistic Regression's confusion matrix shows 115 true negatives, 123 false positives, 59 false negatives, and 183 true positives. It catches actual wins really well, but it's clearly leaning toward predicting wins overall, it's got more than double the false positives compared to false negatives.



Random Forest shows 129 true negatives, 109 false positives, 78 false negatives, and 164 true positives. Same general pattern, more false positives than false negatives, but noticeably more balanced than Logistic Regression's split.

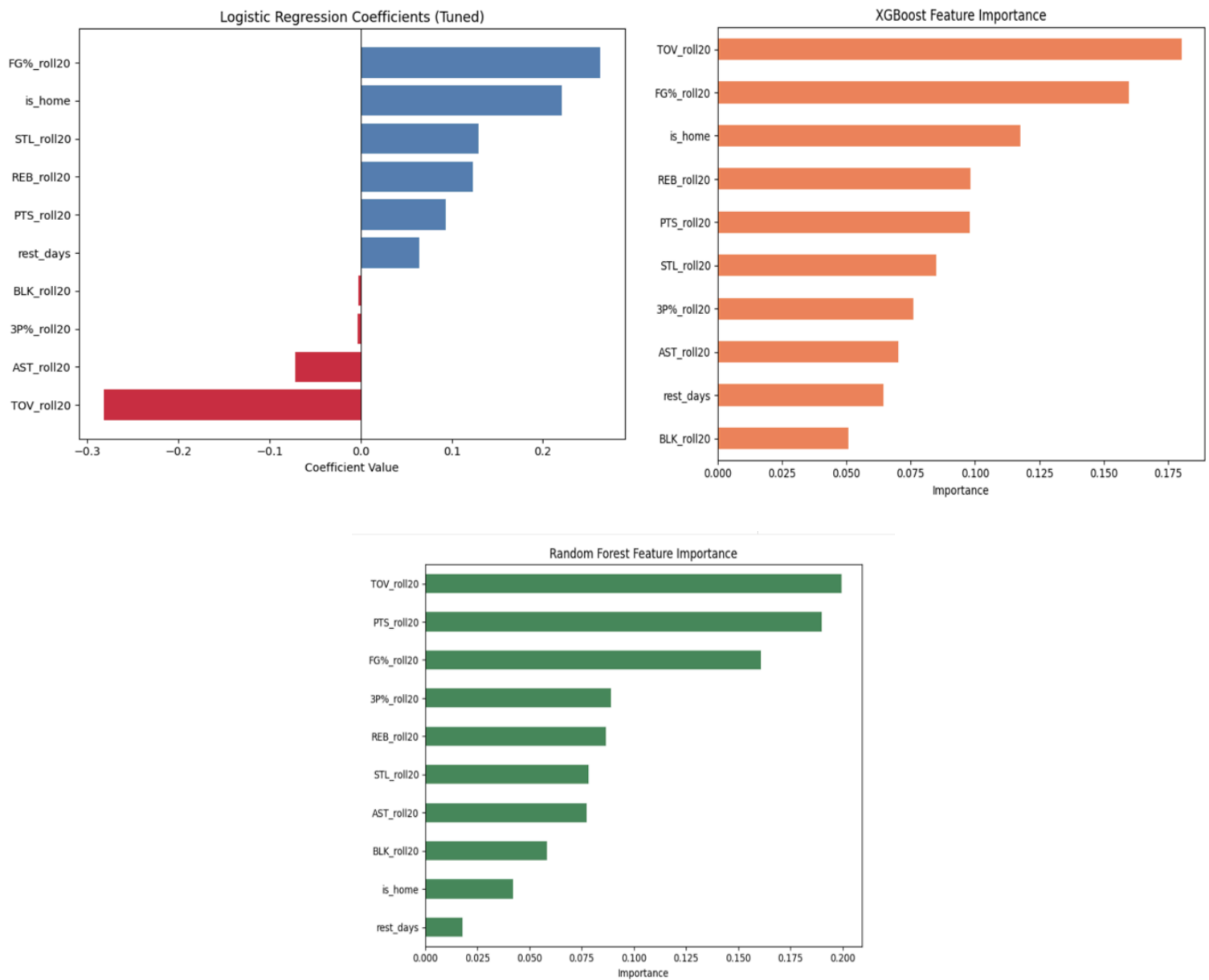


XGBoost lands in between, 124 true negatives, 114 false positives, 71 false negatives, and 171 true positives. Still more false positives than false negatives, but closer to Random Forest's balance than Logistic Regression's

All three models made more false positives than false negatives, so they shared the same tendency to over-predict wins. Logistic Regression had the strongest imbalance between the two error types, while Random Forest was the most balanced. Despite these differences, the overall

pattern was consistent across all three models, suggesting that the feature set may be contributing to this behavior rather than the choice of model alone.

Feature importance plots:



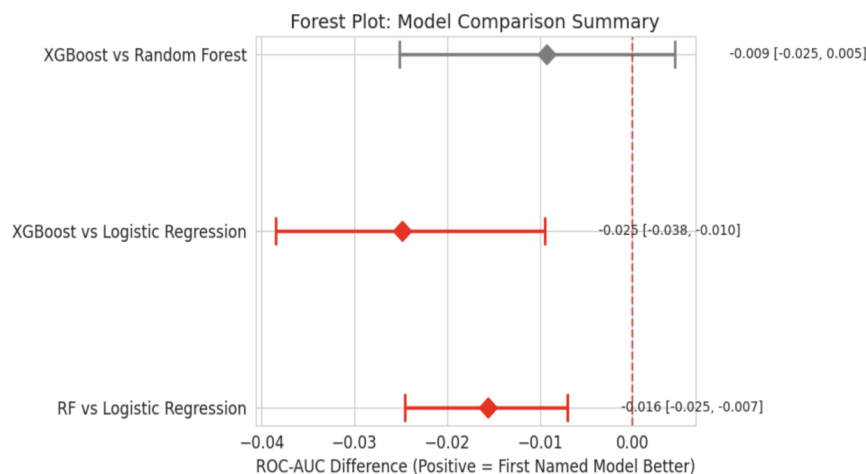
When looking at the feature importance plots, there is a lot of agreement across the three models. For Logistic Regression, turnovers had the strongest effect, with a negative relationship, followed by field goal percentage and then home-court advantage. Random Forest ranked turnovers, field goal percentage, and points as its top three features. XGBoost showed a similar

pattern, with turnovers and field goal percentage among its strongest predictors. Overall, all three models agree that turnovers and shooting efficiency are important for predicting wins, even though they approach the problem differently. One difference is home-court advantage, which was much more important for Logistic Regression but ranked much lower for the two tree-based models.

Statistical Analysis: I used three statistical tests to compare Logistic Regression with Random Forest and XGBoost: Bootstrap confidence intervals, Wilcoxon signed-rank, and McNemar's test. The Bootstrap results favored Logistic Regression in both comparisons. The mean ROC-AUC difference was -0.0156 for Random Forest versus Logistic Regression (95% CI: [-0.0246, -0.0071]) and -0.0248 for XGBoost versus Logistic Regression (95% CI: [-0.0385, -0.0095]). Both confidence intervals were entirely below zero, indicating a statistically significant advantage for Logistic Regression based on the primary test. The XGBoost versus Random Forest comparison was not significant (difference = -0.0093, 95% CI: [-0.0252, 0.0046]).

The Wilcoxon test did not find a significant difference for any comparison ($p = 0.0625$, 0.125 , and 0.4375), and McNemar's test also showed no significant differences ($p = 0.644$, 0.824 , and 0.877). These tests may have had limited power because only five cross-validation folds were available. Overall, the primary Bootstrap analysis supports the conclusion that Logistic Regression significantly outperformed both Random Forest and XGBoost.

The forest plot below visualizes these bootstrap confidence intervals for all three model comparisons:

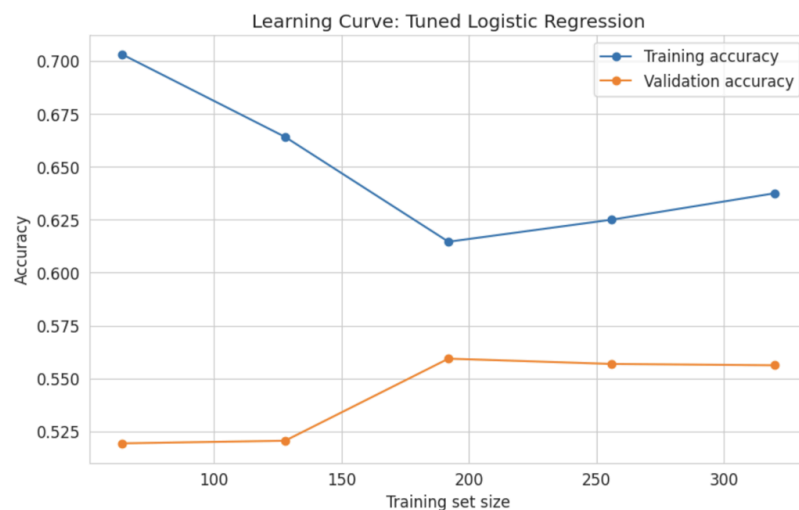


Error Analysis: The tuned Logistic Regression model correctly predicted 298 of 480 test observations (62.1% accuracy). It was better at identifying actual wins, correctly predicting 183 of 242 wins (75.6% recall), but was less reliable when predicting a win, with 59.8% precision. This resulted in 123 false positives compared with 59 false negatives.

Looking more closely at the misclassified examples, FG%_roll20 stood out as the model's strongest predictor (coefficient = +0.263). In 7 of the 8 examples reviewed, the model predicted a win when FG%_roll20 was above the league average and a loss when it was below. However, this simple pattern still led to incorrect predictions in both directions, suggesting that recent shooting performance alone is not enough to accurately predict every game outcome.

Interpretation: My original hypothesis, that Random Forest and XGBoost would outperform Logistic Regression, was refuted. Based on the primary Bootstrap confidence interval analysis, Logistic Regression significantly outperformed both alternative models on ROC-AUC, which was the opposite of what I expected.

One possible explanation is that the relationships captured by my features are relatively simple and do not benefit from the added complexity of tree-based models. Random Forest and XGBoost are designed to capture non-linear relationships and feature interactions, but that advantage only helps when those patterns provide meaningful additional information. The feature importance results support this interpretation, as all three models identified turnovers, field goal percentage, and points as their strongest predictors. This suggests that the more complex models were not finding much additional signal beyond what Logistic Regression was already able to capture.



To determine whether the results were influenced by overfitting or underfitting, I examined the learning curve for the tuned Logistic Regression model. Training accuracy started around 0.70 with a small training set, decreased to about 0.615 as more data was added, and then increased slightly to about 0.64. Validation accuracy increased steadily from about 0.52 and leveled off around 0.556–0.560. The gap between training and validation accuracy also narrowed from roughly 0.18 to 0.06–0.08 as the training set grew. This pattern could also suggest mild overfitting. Overall, the results suggest that the model is learning a real but limited pattern that generalizes reasonably well.

Results:

Strengths: I tested five rolling window sizes (5, 10, 15, 20, and 25 games) and selected 20 games based on model performance rather than assumption. I also used a persistence baseline, which was more appropriate for the time-series structure than a majority-class or random-guess baseline given the balanced classes. All three models underwent hyperparameter tuning using multiple scoring metrics, and I used three statistical tests (Bootstrap CI, Wilcoxon, and McNemar's) to strengthen the comparison rather than relying on a single test.

Limitations: The feature set only captures a team's recent performance and does not include opponent strength, so the model cannot account for matchup difficulty. The features also have some mild multicollinearity, which is expected with correlated basketball statistics but can affect how individual coefficients are interpreted. Finally, using only the 2025 season limits how well the results may generalize to other seasons, rules, and rosters.

Biases: Box scores are reliable, but they do not capture context such as injuries, lineup changes, or coaching decisions. The rolling features also do not account for trades, injuries, or other roster changes, so recent statistics may sometimes reflect a different roster. Finally, I did not test performance across teams, conferences, or team quality levels, so the model may perform differently across these groups.

Societal Impact: In practice, a model like this could support fan analytics, broadcast commentary, or identifying performance trends. The main concern would be using it for sports betting, where its modest performance ($\text{ROC-AUC} \approx 0.66$) could create false confidence in decisions involving real money.

Takeaway/ lessons:

- Testing choices like rolling window size with actual data is more reliable than choosing them based on intuition.
- More complex models do not always perform better. When different models identify the same important features, the data may be the main limitation rather than the algorithm.
- Using multiple statistical tests showed that different methods can lead to different conclusions, making it important to understand why the results differ.

V. Conclusion

This project examined whether recent team performance could help predict NBA game outcomes and whether Random Forest and XGBoost would outperform Logistic Regression. The hypothesis was refuted. Based on the primary Bootstrap confidence interval analysis, Logistic Regression significantly outperformed both tree-based models on ROC-AUC. However, all three models outperformed the persistence baseline, showing that recent team performance does contain some predictive signal.

Feature importance and error analysis showed that all three models relied on similar predictors, particularly turnovers, field goal percentage, and points. This suggests that the added complexity of Random Forest and XGBoost did not provide much additional predictive value. The main limitation was the feature set, especially the lack of opponent-strength information, which limited the model's ability to account for matchup difficulty.

Future work should focus on adding opponent-strength features, such as recent opponent win percentage or rolling opponent statistics, and testing the models across multiple seasons. It would also be useful to test rolling window sizes separately for Random Forest and XGBoost rather than assuming the 20-game window selected for Logistic Regression is optimal for all models.

VI. Reproducibility and References

Data Source:

NBA Traditional Box Scores dataset. Kaggle.

<https://www.kaggle.com/datasets/szymonjwiak/nba-traditional/data?select=traditional.csv>

Libraries and Tools:

LogisticRegression: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

RandomForestClassifier:<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>

GridSearchCV:https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html

TimeSeriesSplit:https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html

pandas, Numpy, matplotlib, seaborn, scipy

I used an LLM (Claude, Anthropic) throughout this project to come up with my window-size parameter testing approach, debug code, verify statistical test implementations, and check my interpretation of results